HELIOS Processor

Public API

v25.11.0.23637
2025-12-01

HELIOS Processor - Public API

Copyright © Megapixel

The Megapixel logo is a trademark of H2VR HoldCo, Inc.

Other trademarks and trade names may be used in this document to refer to products by other entities. H2VR HoldCo, Inc. claims no proprietary interest in trademarks and trade names owned by others.

Information and specifications in this document are subject to change without notice. H2VR HoldCo. assumes no responsibility or liability for any errors or inaccuracies that may appear in this manual.

Contact:

https://www.megapixelvr.com

Document History

Date	Author	Notes
2016/11/28	CO	Initial version
2016/12/5	CO	Updated with correct JSON
2016/12/21	CB	Described preview retrieval
2017/1/4	CO	Reviewed layout
2017/2/1	CO	Refactoring of some display properties, added IP gateway settings & more info on alerts.
2017/3/11	CO	Updated after release of Sprint #20: added authentication.
2017/4/4	CO	Added more elaborate information about alerts
2017/5/31	CB	Further elaborated on alerts
2018/1/29	CB	Reformatted document
2020/2/24	SB	Updated product/company names and deprecating dev.Distro
2020/4/26	CB	dev.ingest.input is R/W
2020/7/31	CB	Correct preview URL
2021/1/5	CB	Saved configurations
2021/9/27	CB	Test patterns, additional preview sources
2022/1/29	CB	dev.display.out.adjust, dev.groups
2022/2/25	CB	dev.display.redundancy, removed dev.distros
2022/8/11	CB	Still store
2022/9/17	CB	Basic auth
2022/12/6	CB	Uploading to still store
2023/3/16	CB	Stop showing a still
2023/8/24	CB	dev.display.cctDuv
2023/9/19	CB	Blackout example
2023/12/16	CB	List of saved configurations /api/v1/presets/list
2024/5/3	CB	Brief description of HTTP
2024/8/16	CB	Fill in data model property types, added group examples
2024/8/23	CB	Camera/GhostFrame configuration /api/v1/camera. Tile controls /api/v1/tile
2025/3/29	CB	Group management /api/v1/groups
2025/4/17	CB	Discovery
2025/7/16	CB	Select input example
2025/8/7	CB	Processor reboot

Table of Contents

1. Introduction
2. Discovery
   - 2.1. mDNS DNS-SD
   - 2.2. UPNP SSDP
3. Accessibility
   - 3.1. HTTP - Hypertext Transfer Protocol
     - 3.1.1. GET request
     - 3.1.2. POST request
     - 3.1.3. Malformed requests
     - 3.1.4. Missing content-type
4. REST Protocol
   - 4.1. Property interface `/api/v1/public`
     - 4.1.1. Querying properties
       - 4.1.1.1. Retrieve all properties
       - 4.1.1.2. Retrieve specific properties
     - 4.1.2. Changing properties
     - 4.1.3. Setting specific properties
     - 4.1.4. Error conditions
     - 4.1.5. Method not allowed
     - 4.1.6. Expected application/json Content-Type
   - 4.2. Camera/GhostFrame® controls `/api/v1/camera`
     - 4.2.1. Retrieve current settings
     - 4.2.2. Adjusting camera settings
       - 4.2.2.1. Enable camera mode
       - 4.2.2.2. Set slices
   - 4.3. Displaying solid colors `/api/v1/color/pattern` (for use by automated measuring tools)
     - 4.3.1. How is this different than HELIOS' test pattern interface?
     - 4.3.2. Showing solid 70% red (non-linear space)
     - 4.3.3. Stop showing solid colors
   - 4.4. Group management `/api/v1/groups`
     - 4.4.1. List groups
     - 4.4.2. Retrieve a specific group
     - 4.4.3. Create a group
     - 4.4.4. Update a group
     - 4.4.5. Delete a group
   - 4.5. Still store `/api/v1/media`
     - 4.5.1. List available stills
     - 4.5.2. Show a still (by id)
     - 4.5.3. Show a still (by name)
     - 4.5.4. Stop showing a still
     - 4.5.5. Uploading a still
     - 4.5.6. Example - Showing a still by id (id = 26)
     - 4.5.7. Example - Show still by name (name = "VGA-no-signal-image.jpeg")
   - 4.6. Preview interface `/api/v1/preview`
     - 4.6.1. Sources
     - 4.6.2. MJPEG
   - 4.7. Saved configurations `/api/v1/presets`
     - 4.7.1. List of available configurations (names)

4.7.2. List of available configurations (full)
4.7.3. Applying a saved configuration (by id)
4.7.4. Applying a saved configuration (by presetName)
4.7.5. Example - Applying the saved configuration by id (id = 3)
4.7.6. Example - Apply saved configuration by name (presetName = "Global Settings")
4.8. Tile controls `/api/v1/tile`
4.8.1. Capture to tile's still store `/api/v1/tile/captureStill`
4.9. More info
4.10. System `/api/v1/sys`
4.10.1. System reboot `/api/v1/sys/reboot`
5. JSON-RPC Protocol
5.1. Retrieve all properties
5.2. Retrieve a specific property
5.3. Set a specific property
5.3.1. Error conditions
5.4. Property change notification
5.5. Ping method
5.6. More info
6. Authentication
6.1. Example of a GET request with basic access authentication, using Curl
6.2. Example of a GET request with digest access authentication, using Postman
6.3. More info
7. Data model (JSON)
7.1. `dev` object (devices)
7.1.1. `dev.display`
7.1.1.1. `dev.display.out`
7.1.1.2. `dev.display.out.adjust`
7.1.1.3. `dev.display.redundancy`
7.1.2. `dev.groups`
7.1.3. `dev.ingest`
7.1.4. `dev.receivers`
7.1.5. `dev.receivers.info`
7.2. `sys` object (system)
8. System health
8.1. Overall health status
8.1.1. System level alerts
8.2. Alerts
8.2.1. Severity
8.2.2. Possible alert sources
8.2.3. Since
8.2.4. Example - Over temperature alert (receiver card)
8.2.5. Example - Over temperature alert (Processor)
8.2.6. Example - Port capacity (switch + receiver)
9. Appendix
9.1. Data operations overview
9.2. Example - Blackout
9.2.1. Blackout
9.2.2. Un-blackout
9.3. Example - Control screen brightness
9.3.1. Get screen brightness
9.3.2. Set screen brightness

9.4. Example - Control screen gamma
9.4.1. Get screen gamma
9.4.2. Set screen gamma
9.5. Example - Control test patterns
9.5.1. Enable color bars test pattern
9.5.2. Show solid red test pattern
9.5.3. Disable test pattern
9.6. Example - Control inputs
9.6.1. List available inputs
9.6.2. Current input
9.6.3. Select input
9.7. Example - Redundancy flip to main/backup
9.7.1. Flip to main
9.7.2. Flip to backup
9.8. Example - Groups
9.8.1. Group blackout
9.8.2. Group 50% intensity

1. Introduction

Megapixel's HELIOS Processors are equipped with a 'public API' which allows third party systems to interface with a Processor, connected downstream distribution switches, and display devices. This document will explain how you can use this public API and will supply an overview of the possibilities to query and control a HELIOS Processor system, from an external system.

This document requires knowledge of common web technologies like URL, HTTP, JSON and REST interfaces.

2. Discovery

HELIOS by default is network discoverable via the mDNS DNS-SD and UPNP SSDP protocols. However, these services may be disabled via Processor Settings to address security concerns.

2.1. mDNS DNS-SD

HELIOS advertises an _http._tcp. service referencing HELIOS web interface address, along with the following txt fields:

- description - User provided description
- model - HELIOS
- manufacturer - Megapixel
- serial - HELIOS' serial number
- version - Software version
- role - Redundancy role (e.g., main)
- stack - Stack name

2.2. UPNP SSDP

HELIOS exposes the following device fields:

- friendlyName - User provided description
- manufacturer - Megapixel
- modelName - HELIOS
- serialNumber - HELIOS' serial number
- presentationURL - Web interface address

3. Accessibility

The public API of a Processor can be reached using the following URL:

> http://IP_ADDRESS/api/v1/public

Where IP_ADDRESS is the IP address of the Processor (same address used to access the processor's web application).

The public API can be accessed using two protocols:

1. HTTP REST web service
2. HTTP WebSocket JSON-RPC

3.1. HTTP - Hypertext Transfer Protocol

HTTP is a request/response protocol over a TCP connection.

Port: TCP 80 (HTTP - non-encrypted)
Port: TCP 443 (HTTPS - encrypted)

- Requests are composed of a request line, followed by header fields, a blank line, and content body.
- Lines are terminated with a carriage return + line feed (\r\n).
- Refer to RFC 1945 (HTTP/1.0), RFC 9112 (HTTP/1.1), and related RFCs for more details

NOTE: Remainder of this document only specifies the METHOD and URL for API requests. While all requests are expected to comply with HTTP.

3.1.1. GET request

Request (using a GET method, public API endpoint, HTTP version 1.0, without header fields, and no body):

```
GET /api/v1/public HTTP/1.0


```

(blank line after the headers is required, and GET requests have no content body)

Response:

```
HTTP/1.1 200 OK
X-Content-Type-Options: nosniff
Cache-Control: private, no-cache, no-store, must-revalidate
Pragma: no-cache
X-RD-Uptime: 7204.7
Content-Type: application/json; charset=utf-8
Content-Length: 47545
Date: Fri, 03 May 2024 03:23:50 GMT
Connection: close

{"dev":{"display":{"blackClip  ...
```

3.1.2. POST request

```
POST /api/v1/public HTTP/1.0
Content-Type: application/json
Content-Length: 37

{"dev":{"display":{"blackout":true}}}

HTTP/1.1 200 OK
X-Content-Type-Options: nosniff
Cache-Control: private, no-cache, no-store, must-revalidate
Pragma: no-cache
Content-Type: application/json; charset=utf-8
Content-Length: 37
Date: Fri, 03 May 2024 03:30:40 GMT
Connection: close

{"dev":{"display":{"blackout":true}}}
```

3.1.3. Malformed requests

HELIOS will typically respond with a 400 status when the request is malformed.

```
malformed request

HTTP/1.1 400 Bad Request
Connection: close
```

3.1.4. Missing content-type

POST requests will respond with a 406 status when a JSON encoded content body is expected but Content-Type hasn't be set to `application/json`.

```
POST /api/v1/public HTTP/1.0
Content-Length: 37

{"dev":{"display":{"blackout":true}}}

HTTP/1.1 406 Not Acceptable
Content-Type: text/plain; charset=utf-8
Content-Length: 38
Date: Fri, 03 May 2024 03:45:29 GMT
Connection: close

Expected application/json Content-Type
```

4. REST Protocol

The REST web service uses JSON to represent the data and uses the standard HTTP methods like GET & PATCH to read or write properties.

4.1. Property interface `/api/v1/public`

4.1.1. Querying properties

```
GET /api/v1/public?[property[&...]]
```

Query	Description
_	(Optional) Ignored parameter for cache busting (e.g,. `GET /api/v1/public?_=1480466838`)
property	(Optional) Property path to query (e.g., `dev.display.brightness`)
...	(Optional) Additional property paths (e.g., `dev.display.brightness&dev.display.gamma`)

4.1.1.1. Retrieve all properties

Request:

```
GET /api/v1/public
```

Response:

```
{
    "dev": {
        "display": {
            "brightness": 50,
            "gamma": 1.5,
            "...": "..."
        }
    },
    "...": "..."
}
```

4.1.1.2. Retrieve specific properties

Request brightness & gamma:

```
GET /api/v1/public?dev.display.brightness&dev.display.gamma
```

Response:

```
{
    "dev": {
        "display": {
            "brightness": 50,
            "gamma": 2.4
        }
    }
}
```

4.1.2. Changing properties

```
PATCH /api/v1/public

{
    "path": "new value"
}
```

Query	Description
_	(Optional) Ignored parameter for cache busting (e.g,. `GET /api/v1/public?_=1480466838`)

4.1.3. Setting specific properties

Request to set brightness & gamma:

```
PATCH /api/v1/public

{"dev": {"display": {"brightness": 42, "gamma": 2.2}}}
```

Response:

```
{
    "dev": {
        "display": {
            "brightness": 42,
            "gamma": 2.200000047683716
        }
    }
}
```

4.1.4. Error conditions

Values outside of the supported range will result in the property retaining its current value

Request:

```
PATCH /api/v1/public

{"dev": {"display": {"brightness": 1000}}}
```

Response:

```
{
    "dev": {
        "display": {
            "brightness": 42
        }
    }
}
```

Unsupported or unknown properties are ignored

Request:

```
PATCH /api/v1/public

{"dev": {"display": {"invalid": true}}}
```

Response:

```
{
    "dev": {
        "display": {}
    }
}
```

4.1.5. Method not allowed

Occurs when the requested URL is malformed. When PATCHing the public API check that the URL in the request is set correctly (`/api/v1/public`).

4.1.6. Expected application/json Content-Type

Most HELIOS' API endpoints expect JSON formatted bodies. HTTP headers should include a 'content-type' header with the value 'application/json'.

4.2. Camera/GhostFrame® controls `/api/v1/camera`

4.2.1. Retrieve current settings

```
GET /api/v1/camera
```


```
{
    "enabled": true,
    "videoStreams": 2,
    "slices": [
        {
            "type": "V1",
            "invert": false,
            "gains": { "r": 1, "g": 1, "b": 1, "i": 1 }
        },
        {
            "type": "V2",
            "invert": false,
            "gains": { "r": 1, "g": 1, "b": 1, "i": 1 }
        },
        {
            "type": "K",
            "invert": false,
            "gains": { "r": 1, "g": 1, "b": 1, "i": 1 }
        }
    ],
    "tracks": {
        "C1": {
            "color": { "r": 1, "g": 0, "b": 0 },
            "gains": { "r": 1, "g": 1, "b": 1, "i": 1 }
        },
        "C2": {
            "color": { "r": 1, "g": 0, "b": 1 },
            "gains": { "r": 1, "g": 1, "b": 1, "i": 1 }
        },
        "K": { "gains": { "r": 1, "g": 1, "b": 1, "i": 1 } },
        "S1": { "gains": { "r": 1, "g": 1, "b": 1, "i": 1 } },
        "S2": { "gains": { "r": 1, "g": 1, "b": 1, "i": 1 } },
        "V1": { "gains": { "r": 1, "g": 1, "b": 1, "i": 1 } },
        "V2": { "gains": { "r": 1, "g": 1, "b": 1, "i": 1 } },
        "V3": { "gains": { "r": 1, "g": 1, "b": 1, "i": 1 } },
        "V4": { "gains": { "r": 1, "g": 1, "b": 1, "i": 1 } }
    }
}
```


Above response indicates that we are configured for two video streams with three slices. Camera mode is enabled. Slices are V1, V2, and K. Gains are all unity.

When compared to HELIOS' Camera/GhostFrame® interface. Slices are vertical columns, while tracks are the horizontal row. i.e., gains in the V1 track will apply to all slices referencing V1.

Color patches C1 & C2 have an additional `color` field. These are currently normalized 8 bit values. Where a value of 1.0 = 255. While both color & gains equate to the same controls, both are exposed here for parity with HELIOS' UI.

4.2.2. Adjusting camera settings

Most settings can be patched (e.g., just providing `enabled`). Exception to this case is `slices`, which must be fully specified as it will overwrite the existing slice & slice gains configuration.

Refer to the track names as a list of supported slice types. Unsupported types will be accepted but skipped during frame generation. Convention is to use `X` for an unassigned slice.

Similarly specifying a video stream beyond the supported number of streams (e.g., `V9`) will also be skipped during frame generation.

For example a slice configuration of [V1, V2, X, V9] is equivalent to [V1, V2] as X and V9 are skipped.

While the `invert` property is available on each slice, this setting will be ignored when HELIOS lacks a GhostFrame® license.

4.2.2.1. Enable camera mode

```
PATCH /api/v1/camera
content-type: application/json

{ "enabled": true }
```

Response:

```
{
    "enabled": true,
    ... rest of state ...
}
```

4.2.2.2. Set slices

```
PATCH /api/v1/camera
content-type: application/json

{
    "slices": [
        { "type": "V2" },
        { "type": "V2" }
    ]
}
```

Response:

```
{
    "slices": [
        {
            "type": "V2",
            "invert": false,
            "gains": { "r": 1, "g": 1, "b": 1, "i": 1 }
        },
        {
            "type": "V2",
            "invert": false,
            "gains": { "r": 1, "g": 1, "b": 1, "i": 1 }
        }
    ],
    ... rest of state ...
}
```

(as we haven't defined slice gains, they will be reset to unity)

4.3. Displaying solid colors `/api/v1/color/pattern` (for use by automated measuring tools)

`/api/v1/color/pattern` exposes normalized color coefficients which can be used to augment/replace incoming video pixel data. This is for use by automated measuring tools needing to show a range of specific solid color values on the display.

As these color coefficients are normalized, they will automatically be expanded out to the maximum bit depth supported by HELIOS' & PX1 color pipelines. This is independent of the active video's bit depth & quantization range, but will be similarly affected by low brightness levels, and other system adjustments.

An alpha value is available to adjust the mix of specified color to input pixel values. Ranging from one - fully opaque (only show pattern color), to zero - transparent (show incoming pixel data only). Recommend leaving alpha as a value of one.

Solid colors are applied in non-linear space, before any artistic adjustments, EOTF, and/or 3D LUTs.

4.3.1. How is this different than HELIOS' test pattern interface?

This solid color interface is designed for use by automated measuring tools. Allowing the color to be overridden regardless of the selected input (including HELIOS' test patterns), minimizing disruption to existing system configurations.

For non-measuring use cases; we recommend preferring HELIOS' test pattern interface (`dev.ingest.testPattern`).

4.3.2. Showing solid 70% red (non-linear space)

```
POST /api/v1/color/pattern
Content-Type: application/json

{
    "enabled": true,
    "r": 0.7,
    "g": 0,
    "b": 0,
    "a": 1
}
```

4.3.3. Stop showing solid colors

```
POST /api/v1/color/pattern

{
    "enabled": false
}
```

OR

```
DELETE /api/v1/color/pattern
```

4.4. Group management `/api/v1/groups`

4.4.1. List groups

```
GET /api/v1/groups
```

Responds with a collection of group records. For example:


```
{
    "groups": [
        {
            "id": 0,
            "blackout": false,
            "gains": { "b": 1, "g": 1, "i": 1, "r": 1 },
            "mask": {
                "enabled": false,
                "offset": { "b": 0, "l": 0, "r": 0, "t": 0 }
            },
            "name": "Ceiling",
            "testPattern": { "a": 1,"b": 1,"enabled": false,"g": 1,"r": 1 }
        },
        {
            "id": 2,
            "blackout": false,
            "gains": { "b": 1, "g": 1, "i": 1, "r": 1 },
            "mask": {
                "enabled": false,
                "offset": { "b": 0, "l": 0, "r": 0, "t": 0 }
            },
            "name": "group 3",
            "testPattern": { "a": 1, "b": 1, "enabled": false, "g": 1, "r": 1 }
        }
    ]
}
```


Where each record contains:

Name	Type	Information
id	Number	Record's id
blackout	Boolean	Group's blackout state
gains	Object	RGB + Intensity
mask	Object	Mask settings
name	String	User assigned name
testPattern	Object	Test pattern settings

4.4.2. Retrieve a specific group

```
GET /api/v1/groups/2
```

Response:

```
{
    "group": {
        "id": 2,
        "blackout": false,
        "gains": { "b": 1, "g": 1, "i": 1, "r": 1 },
        "mask": {
            "enabled": false,
            "offset": { "b": 0, "l": 0, "r": 0, "t": 0 }
        },
        "name": "group 3",
        "testPattern": { "a": 1, "b": 1, "enabled": false, "g": 1, "r": 1 }
    }
}
```

4.4.3. Create a group

Request group creation:

```
POST /api/v1/groups
content-type: application/json

{
    "group": {
        "name": "New group"
    }
}
```

Response:

```
{
    "group": {
        "id": 4,
        "blackout": false,
        "gains": { "b": 1, "g": 1, "i": 1, "r": 1 },
        "mask": {
            "enabled": false,
            "offset": { "b": 0, "l": 0, "r": 0, "t": 0 }
        },
        "name": "New group",
        "testPattern": { "a": 1, "b": 1, "enabled": false, "g": 1, "r": 1 }
    }
}
```

- Group settings MAY be provided to initialize the new group
- Additionally MAY provide a desired `id`, although server will return a 409 if that id is already in use

4.4.4. Update a group

```
PATCH /api/v1/groups/4
content-type: application/json

{
    "group": {
        "blackout": true
    }
}
```

Response:

```
{
    "group": {
        "id": 4,
        "blackout": true,
        "gains": { "b": 1, "g": 1, "i": 1, "r": 1 },
        "mask": {
            "enabled": false,
            "offset": { "b": 0, "l": 0, "r": 0, "t": 0 }
        },
        "name": "New group",
        "testPattern": { "a": 1, "b": 1, "enabled": false, "g": 1, "r": 1 }
    }
}
```

4.4.5. Delete a group

```
DELETE /api/v1/groups/4
```

Response: 204 on success

4.5. Still store `/api/v1/media`

4.5.1. List available stills

A list of the currently stored stills is retrievable via:

```
GET /api/v1/media
```

Which responds with a collection of records. For example:

```
{
    "media": [
        {
            "id": 26,
            "name": "VGA-no-signal-image.jpeg",
            "media": "525ee11685561e223d65018d170820184b4e284a0c128be5919f043dd117eac1.jpeg",
            "details": "{}",
            "createdAt": "2022-07-15T04:12:44.156Z",
            "updatedAt": "2022-07-15T04:12:44.156Z"
        }
    ]
}
```

Where each record contains:

Name	Type	Information
id	Number	Record's id
name	String	Displayed name for this still
media	String	Internal filename
details	Object / null	String encoded JSON with additional details (e.g., starred)
createdAt	String	Time when this record was created (in server time)
updatedAt	String	Time when this record was last updated (in server time)

4.5.2. Show a still (by id)

Stills may be shown via their numerical id:

```
POST /api/v1/media/:id/show
```

4.5.3. Show a still (by name)

Stills may also be shown by name:

```
POST /api/v1/media/show

{
    "name": "{name}"
}
```

Note: That the first matching still will be used in the event that there are multiple stills sharing the same name.

4.5.4. Stop showing a still

```
POST /api/v1/media/hide
```

or show a null name:

```
POST /api/v1/media/show

{
    "name": null
}
```

or delete:

```
DELETE /api/v1/media/show
```

4.5.5. Uploading a still

Stills can be uploaded via POSTing a multipart/form-data with a single "file" value containing the image. For example:

```
POST /api/v1/media
...
Content-Type: multipart/form-data; boundary=------------------------7ae41882d286e32b

--------------------------7ae41882d286e32b
Content-Disposition: form-data; name="file"; filename="103.png"
Content-Type: image/png
...
```

Curl equivalent:

```
curl http://helios-ip/api/v1/media -F file=@103.png
```

Which responds with a record:

```
{
    "type": "media",
    "id": 75,
    "attributes": {
        "id": 75,
        "name": "103.png",
        "media": "3678f3fbc7cd7c3ba83da5e33848a43912aef6affbc9255bffd287d40d32b839.png",
        "updatedAt": "2022-11-28T19:46:48.329Z",
        "createdAt": "2022-11-28T19:46:48.329Z"
    }
}
```

4.5.6. Example - Showing a still by id (id = 26)

```
POST /api/v1/media/26/show
```

Which responded with a 204 (No Content)

Attempts at showing an invalid still will return 404:

```
POST /api/v1/media/9999/show
```

```
Not Found
```

4.5.7. Example - Show still by name (name = "VGA-no-signal-image.jpeg")

```
POST /api/v1/media/show

{
    "name": "VGA-no-signal-image.jpeg"
}
```

Which responded with 204 (No Content)

4.6. Preview interface `/api/v1/preview`

A preview of the currently selected input is available using the following URL:

> http://{IP_ADDRESS}/api/v1/preview/preview

Where IP_ADDRESS is the IP address of the Processor (same address used to access the web application of the Processor).

A GET request for `preview` will return a scaled JPEG image of the currently selected input.

- Preview images are updated periodically (e.g., every half a second)
- If there is no input or the current input is invalid the preview image will be black
- It is possible for the server to respond with a 404 in cases where there is an internal error retrieving a preview. This typically happens in certain debugging scenarios, or when one or more system components have just booted. It is recommended to retry the preview request after a five second timeout in such cases.
- Previews will return a 403 in cases where the remote preview security settings are enabled (from HELIOS' front panel)

4.6.1. Sources

A array of available preview sources can be retrieved via:

> http://{IP_ADDRESS}/api/v1/preview

Example response:

```
["preview","hdmi","sdi1"]
```

Where a specific source can be retrievable via:

> http://{IP_ADDRESS}/api/v1/preview/{SOURCE}

For example:
`http://{IP_ADDRESS}/api/v1/preview/sdi1`

4.6.2. MJPEG

Clients may request a motion JPEG stream:

> http://{IP_ADDRESS}/api/v1/preview/{SOURCE}?mjpeg

JPEG frames are then delivered to the requesting client every half a second.

It is recommended to use the MJPEG stream when possible. As an example; the Web Application will always attempt to use the MJPEG stream. If the MJPEG stream fails, then the (static) previews will be retrieved every three seconds.

4.7. Saved configurations `/api/v1/presets`

4.7.1. List of available configurations (names)

Available: **HELIOS 23.12.1**

A list of the currently saved configurations (AKA presets) is retrievable via:

```
GET /api/v1/presets/list
```

Which responds with a collection of records, for example:

```
{
    "presets": [
        {
            "id": 3,
            "presetName": "Global Settings",
            "createdAt": "2021-01-05T20:36:42.227Z",
            "updatedAt":"2021-01-05T20:36:42.227Z"
        }
    ]
}
```

This collection does not include the underlying configuration data ("filterSettings" & "settings"), making this much more efficient when only the available configuration names is desired.

4.7.2. List of available configurations (full)

A complete list of the currently saved configurations (AKA presets) along with all of the associated data is retrievable via:

```
GET /api/v1/presets
```

Which responds with a collection of records, for example:

```
{
    "presets": [
        {
            "id": 3,
            "presetName": "Global Settings",
            "filterSettings": "[\"brightness\",\"cct\",\"gamma\",\"test-pattern\",\"blackout\"]",
            "settings":"{ ... complete state ... }",
            "createdAt": "2021-01-05T20:36:42.227Z",
            "updatedAt":"2021-01-05T20:36:42.227Z"
        }
    ]
}
```

Where each record contains:

Name	Type	Information
id	Number	Record's id
presetName	String	Saved configuration's name
filterSettings	String	JSON encoded array of settings types that were saved
settings	String	JSON dumped collection of complete system state
createdAt	String	Time when this record was created (in server time)
updatedAt	String	Time when this record was last updated (in server time)

4.7.3. Applying a saved configuration (by id)

Saved configurations may be applied via their numerical id:

```
POST /api/v1/presets/:id/apply
```

Where a successful response will return the applied state.

4.7.4. Applying a saved configuration (by presetName)

Saved configurations may also be applied by presetName:

```
POST /api/v1/presets/apply

{
    "presetName": "{presetName}"
}
```

Where a successful response will return the applied state.

Note: That the first matching preset will be used in the event that there are multiple saved configurations sharing the same presetName.

4.7.5. Example - Applying the saved configuration by id (id = 3)

```
POST /api/v1/presets/3/apply
```

Which responded with:

```
{
    "dev": {
        "display": {
            "brightness": 5.7,
            "cct": 6504,
            "gamma": 2.4,
            "blackout": false
        },
        "ingest": {
            "testPattern": {
                "enabled": false
            }
        }
    }
}
```

Applying an invalid saved configuration will return 404:

```
POST /api/v1/presets/4/apply
```

```
Not Found
```

4.7.6. Example - Apply saved configuration by name (presetName = "Global Settings")

```
POST /api/v1/presets/apply

{
    "presetName": "Global Settings"
}
```

Which responded with:

```
{
    "dev": {
        "display": {
            "brightness": 5.7,
            "cct": 6504,
            "gamma": 2.4,
            "blackout": false
        },
        "ingest": {
            "testPattern": {
                "enabled": false
            }
        }
    }
}
```

4.8. Tile controls `/api/v1/tile`

4.8.1. Capture to tile's still store `/api/v1/tile/captureStill`

Initiate capture to tile's second still store

```
POST /api/v1/tile/captureStill
content-type: application/json

{
    "still": 2
}
```

4.9. More info

REST clients for testing:

- Postman - https://www.getpostman.com

More information on REST web services:

- http://www.restapitutorial.com
- https://en.wikipedia.org/wiki/Representational_state_transfer

4.10. System `/api/v1/sys`

4.10.1. System reboot `/api/v1/sys/reboot`

Initiate system reboot

```
POST /api/v1/sys/reboot
content-type: application/json

{
    "confirm": true
}
```

Response:

`Rebooting`

5. JSON-RPC Protocol

The JSON-RPC service also uses HTTP web sockets to call methods on the web server. A big advantage of the Web Sockets is that the web server can notify clients when a value has changed.

Direct WebSocket URL for the JSON-RPC protocol is:

> ws://IP_ADDRESS/api/v1/public/rpc/websocket

The calls are described in JSON like below:

5.1. Retrieve all properties

Request all properties:

```
{
    "jsonrpc": "2.0",
    "id": 123,
    "method": "state"
}
```

Response:

```
{
    "jsonrpc": "2.0",
    "id": 123,
    "result": {
        "dev": {
            "display": {
                "brightness": 50,
                "gamma": 1.5,
                "...": "..."
            }
        },
        "...": "..."
    }
}
```

5.2. Retrieve a specific property

With the JSON-RPC protocol it is current not possible to query individual properties. Rather, you can use the state method to get all properties at once, and you will then automatically get notified every time a property changes.

5.3. Set a specific property

Request:

```
{
    "jsonrpc": "2.0",
    "id": 123,
    "method": "set",
    "params": {
        "dev": {"display": {"brightness": 42}}
    }
}
```

Response:

```
{
    "jsonrpc": "2.0",
    "id": 123,
    "result": {
        "dev": {"display": {"brightness": 42}}
    }
}
```

5.3.1. Error conditions

Values outside of the supported range will result in the property retaining its current value

Request:

```
{
    "jsonrpc": "2.0",
    "id": 123,
    "method": "set",
    "params": {
        "dev": {"display": {"brightness": 1000}}
    }
}
```

Response:

```
{
    "jsonrpc": "2.0",
    "id": 123,
    "result": {
        "dev": {"display": {"brightness": 42}}
    }
}
```

Unsupported/Unknown properties are ignored

Request:

```
{
    "jsonrpc": "2.0",
    "id": 123,
    "method": "set",
    "params": {
        "dev": {"invalid": true}
    }
}
```

Response:

```
{
    "jsonrpc": "2.0",
    "id": 123,
    "result": {}
}
```

5.4. Property change notification

Once you are connected to the web service, you will automatically get notifications for all property changes, even when you have not (yet) retrieved the current state of the system. There is no need to subscribe to receive the updates.

Notifications can be for individual properties (like display brightness in case the brightness is changed), or for multiple properties at once (for instance receivercards' x & y properties when all tiles have moved).

Response:

```
{
    "jsonrpc": "2.0",
    "method": "update",
    "params": {
        "dev": {"display": {"brightness": 100}}
    }
}
```

5.5. Ping method

The ping method can be used to test the connection with the web service.

Request:

```
{
    "jsonrpc": "2.0",
    "id": 123,
    "method": "ping"
}
```

Response:

```
{
    "jsonrpc": "2.0",
    "id": 123,
    "result": "pong"
}
```

5.6. More info

More information on the JSON-RPC protocol:

- http://json-rpc.org/
- https://en.wikipedia.org/wiki/JSON-RPC

6. Authentication

HELIOS supports basic access authentication & digest access authentication to authenticate access to its web server. When authentication is enabled, it is enabled for all API access, including the public API.

For authentication you'll need:

- Username: user name as defined in the HELIOS web application
- Password: password as defined in the HELIOS web application

with digest based authentication additionally requiring (at a minimum):

- Realm: always "M8 Processor"
- Nonce: nonce value supplied by the server (in the HTTP header)

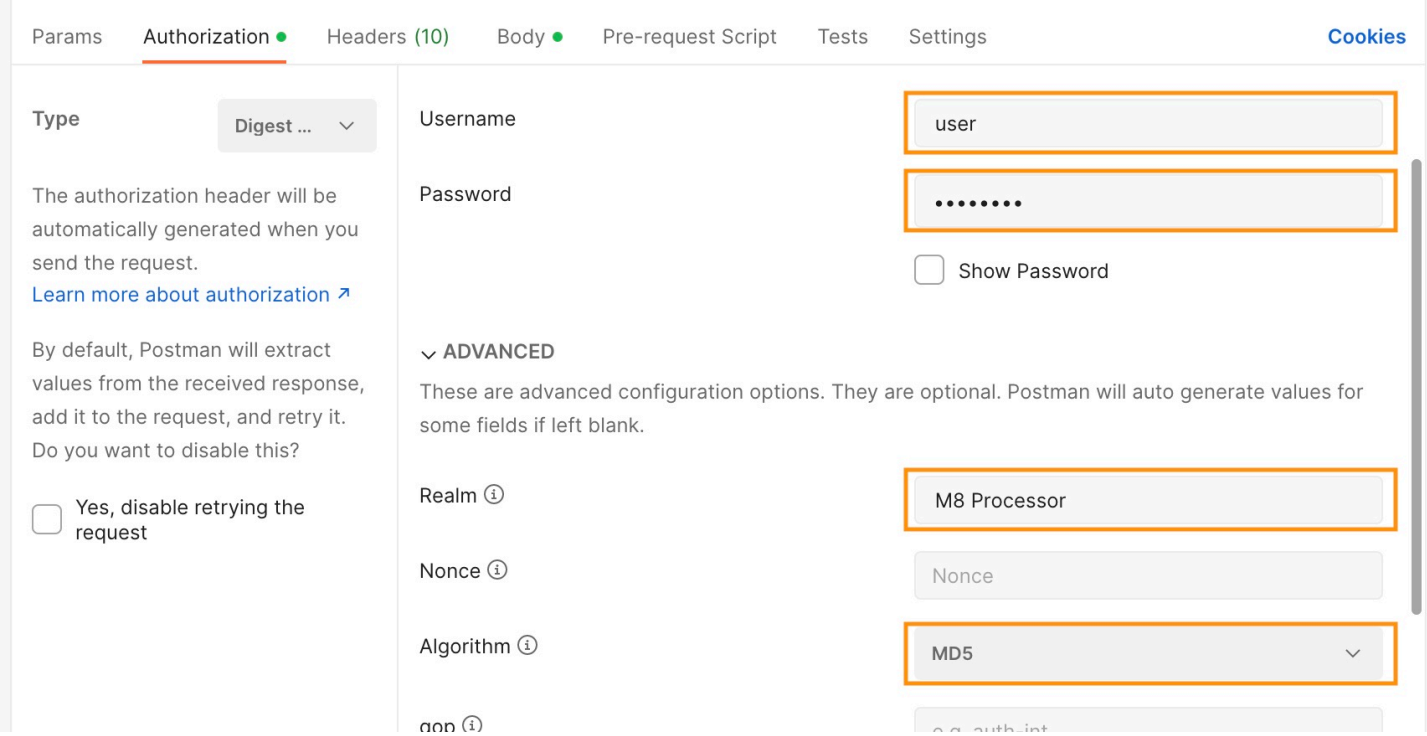
Please read RFC 2617 for more information about the digest access based authentication and how it works.

In cases where network security is a concern; please use https with a trusted certificate to protect credentials.

6.1. Example of a GET request with basic access authentication, using Curl

```
curl http://user:password@{IP_ADDRESS}/api/v1/public
```

6.2. Example of a GET request with digest access authentication, using Postman



6.3. More info

- Basic Access Authentication (Wikipedia): https://en.wikipedia.org/wiki/Basic_access_authentication
- Digest Access Authentication (Wikipedia): https://en.wikipedia.org/wiki/Digest_access_authentication
- RFC 2617: Basic and Digest Access Authentication; https://tools.ietf.org/html/rfc2617

7. Data model (JSON)

The JSON model contains 2 root objects:

- `dev` : device specific properties of the processor and display devices
- `sys` : overall system settings that are specific to a system configuration.

Each of these root objects will be explained in following sections.

7.1. `dev` object (devices)

The dev object keeps track of the device settings and properties.

7.1.1. `dev.display`

This object contains the output or display settings of the HELIOS Processor, including:

Name	Type	R/W	Information
blackout	Boolean	R/W	Black-out status of the display (as boolean)
brightness	Number	R/W	The brightness level of the display (in percentage)
cct	Number	R/W	Color temperature (in Kelvin)
cctDuv	Number	R/W	Color temperature Duv
freeze	Boolean	R/W	Freeze status of the display (as boolean)
gamma	Number	R/W	The gamma correction value
gamut	Object	R	Display gamut details (ideal = widest, target = desired)
out	Object	R/W	Output adjustments
redundancy	Object	R/W	Redundancy information + controls
tilesCount	Number	R	The total number of tiles in the display
tilesInfo	String	R	Information about the tiles in the display
width	Number	R/W	width of the output window
height	Number	R/W	height of the output window
x	Number	R/W	x position of the output window
y	Number	R/W	y position of the output window

7.1.1.1. `dev.display.out`

Name	Type	R/W	Information
adjust	Object	R/W	Output adjustments

7.1.1.2. `dev.display.out.adjust`

Name	Type	R/W	Information
enabled	Boolean	R/W	Output adjustments enabled
gain	Object	R/W	RGB gain
gamma	Object	R/W	RGB gamma
lift	Object	R/W	RGB lift
offset	Object	R/W	RGB offset
saturation	Number	R/W	Output saturation

7.1.1.3. `dev.display.redundancy`

Name	Type	R/W	Information
info	String	R	Processor's overall redundancy state (e.g., "Fail over - Main (Active)")
mode	String	R/W	Redundancy mode (none/failover/seamless)
processors	Object	R	Detected processors organized by UUID
role	String	R/W	Redundancy role (main/backup/offline)
state	String	R/W	Redundancy state (active/mixed/standby). Set to "main"/"backup" to initiate appropriate flip

7.1.2. `dev.groups`

dev.groups object is a dictionary containing all the defined groups within the system. Groups are sequentially indexed but may contain gaps if groups have been removed.

For example: `dev.groups.0`

These group indices are referenced by receivers `.groupId` which defines which group a receiver is a member of.

Name	Type	R/W	Information
blackout	Boolean	R/W	Group's black-out status
gains	Object	R/W	RGBI gains
mask.enabled	Boolean	R/W	Enable/disable group's mask
mask.l/t/b/r	Number	R/W	Mask edge offset
name	String	R/W	User defined name for this group
testPattern.enabled	Boolean	R/W	Enable group's test pattern
testPattern.r/g/b/a	Number	R/W	Group's test pattern color

7.1.3. `dev.ingest`

The dev.ingest object contains the properties for the Processor.

Name	Type	R/W	Information
alerts	Object	R	List of alerts
alertsCount	Number	R	Number of alerts (in alerts)
alertsSeverity	Number	R	Lowest alert severity (-1 when there are no alerts)
color.pattern.enabled	Boolean	R/W	Enable color override
color.pattern.r/g/b/a	Number	R/W	Red/Green/Blue/Alpha color override
info	Object	R	General information like serial number and software/firmware version
input	String	R/W	The selected input
testPattern.enabled	Boolean	R/W	Enable/Disable test pattern
testPattern.motion	Boolean	R/W	Enable/Disable test pattern motion
testPattern.type	String	R/W	Test pattern type (e.g., red, grid, etc)

7.1.4. `dev.receivers`

dev.receivers object contains a dictionary with all the discovered receivers connected to the processor. Receivers are a general term to describe the receiving end of the video data, so these are the display devices or LED panels.

The keys in the dictionary are the MAC addresses of the receivers (display devices).

For example an object for the LED panel with MAC address '58:98:6f:00:01:7d':

> `dev.receivers.58:98:6f:00:01:7d`

Name	Type	R/W	Information
alerts	Object	R	List of alerts
alertsCount	Number	R	Number of alerts (in alerts)
alertsSeverity	Number	R	Lowest alert severity (-1 when there are no alerts)
distroBoxMac	String	R	MAC address of the HELIOS output port to which this receiver is connected
distroBoxPort	Number	R	HELIOS output port to which this receiver is connected
groupId	Number	R/W	Index of group receiver is a member of (-1 = no group)
info	Object	R	General information like name, type, serial number and software/firmware version
switchId	String	R	Key of associated parent switch
switchPort	Number	R	Switch output port to which this receiver is connected
width	Number	R/W	width of this receiver, accounting for 90 degree rotations, and scaling
height	Number	R/W	height of this receiver, accounting for 90 degree rotations, and scaling
x	Number	R/W	x position of this receiver, on the processor's canvas
y	Number	R/W	y position of this receiver, on the processor's canvas

7.1.5. `dev.receivers.info`

Name	Type	R/W	Information
backupConnected	Boolean	R	Tile is connected to the backup processor
connected	Boolean	R	Tile is connected to this processor
name	String	R	Product name
ldmsH	Number	R	Number of LDMs vertically
ldmsW	Number	R	Number of LDMs horizontally
pixelsH	Number	R	Number of pixels vertically
pixelsW	Number	R	Number of pixels horizontally
serial	String	R	Tile's serial number
version	Object	R	Version collection

7.2. `sys` object (system)

The system object contains general system settings that are not specific to a device.

Name	Type	R/W	Information
alerts	Object	R	List of alerts
alertsCount	Number	R	Number of alerts (in alerts)
alertsSeverity	Number	R	Lowest alert severity (-1 when there are no alerts)
description	String	R/W	The name of the Processor (as displayed on the LCD screen and on the web application)
info	Object	R	General information like name, type, serial number and software/firmware version
ipAddress	String	R	Current IP address of the Processor
ipDns	String	R	Current primary DNS address
ipGateway	String	R	Current IP gateway
ipSubnet	String	R	Current IP subnet mask

8. System health

To get the overall health status, look at the alerts properties on the `sys` object.

More detailed alerts are also available at the different device levels (`dev.ingest` , `dev.display` , `dev.switches.[SWITCH]` , and `dev.receivers.[RECEIVER]`).

8.1. Overall health status

An indication of the overall system's health is available via `sys.alertsCount` & `sys.alertsSeverity` .

REST Example

Request health overview:

```
GET /api/v1/public?sys.alertsCount&sys.alertsSeverity
```

Response:

```
{
    "sys":{
        "alertsCount": 2,
        "alertsSeverity": 3
    }
}
```

Informs that the system currently has two alerts, with at least one of those alerts having a severity of Error.

8.1.1. System level alerts

Request to gather system level alerts:

```
GET /api/v1/public?sys.alerts
```

Response:


```
{
    "sys":{
        "alerts":{
            "fanSlow":{
                "brief": "Slow fan",
                "count": 2,
                "desc": "Below normal operating speed",
                "devices":{
                    "receivers":{
                        "58:98:6f:00:0b:7a": ""
                    }
                },
                "severity": 4,
                "since": 6922
            },
            "tempHigh":{
                "brief": "Over temp",
                "count": 1,
                "desc": "Exceeds normal operating temperature",
                "devices": {
                    "receivers":{
                        "58:98:6f:00:0b:7a": ""
                    },
                },
                "severity": 3,
                "since": 9874
            }
        }
    }
}
```


Shows our two alerts are a slow fan and a device is over temperature. Our slow fan has a severity of warning, while the over temperature is considered an error.

8.2. Alerts

Alerts are (temporary) messages to notify about a certain unexpected condition of the system like failing fans, over temperature, exceeded port capacity, ...

Alerts will occur on two levels:

- Device level (`dev.*.alert`): specific alerts for each individual device. This can occur for ingests, switches, receivers, and display.
- System level (`sys.alert`): overview of all the current alerts, system wide.

One or more alerts on one or more devices will be aggregated by the Processor into system level alerts. Because of this, it is best to look for the alerts on the system level and if more information or detail is needed to refer to the alert details of each specific device.

8.2.1. Severity

Each alert type has an associated severity level. These levels correspond to the well-known syslog (RFC5424) severity levels which range from (0) Emergency to (7) Debug. To reduce confusion the Processor uses a subset of these severity levels, while maintaining the severity level numbering.

Value	Severity	Description
2	Critical	Critical conditions
3	Error	Error conditions
4	Warning	Warning conditions
5	Notice	Normal but significant condition

8.2.2. Possible alert sources

Overview of possible alerts and their meaning:

Alert Key	Source	Severity	Description
invalidCal	Receiver	Notice	A panel's calibration data is missing or the data is invalid
fanSlow	Device	Warning	One or more fans inside the panel's cabinet is not working properly
frameError	Receiver	Notice	Video frame error occured (frame drop)
pixelCap	Receiver	Error	Pixel capacity exceeded; a receiver card could not be added to the chain because the total pixel capacity of the processor/switch has been exceeded, although the capacity of the switch port on which the receiver card is connected has not been exceeded
pixelClamp	Receiver	Notice	Pixel clipping or clamping has occurred because of bandwidth issues
portCap	Device	Error	Port capacity exceeded; in case of a receiver card alert, it means the receiver card could not be added to the chain. In case of a switch alert, it means one or more receiver cards could not be added to a switch port because the capacity has been exceeded.
safeMode	Receiver	Notice	Receiver card is in safe mode. In safe mode, the panel's functionality has been limited; the panel can be mapped but no video will be displayed.
tempHigh	Device	Error	The temperature of the device or one of its component has exceeded the maximum value

8.2.3. Since

`since` property reports the time at which an alert was first asserted. This time is measured in seconds since the Processor was started. It is up to the requester to convert this time point to a more meaningful representation.

To aid with the uptime conversion, each HTTP API response from the Processor includes the current Processor uptime in an 'X-RD-Uptime' HTTP response header.

Request:

```
GET /api/v1/public?sys.alerts
```

Response:

```
HTTP/1.1 200 OK
Cache-Control: private, no-cache, must-revalidate
Pragma: no-cache
X-RD-Uptime: 14232
Content-Type: application/json; charset=utf-8
Content-Length: 1948
Vary: Accept-Encoding
Date: Thu, 01 Jun 2017 00:28:49 GMT

{
    "sys": {
        "alerts": {
            "tempHigh":{
                "brief": "Over temp",
                "count": 1,
                "desc": "Exceeds normal operating temperature",
                "devices": {
                    "receivers": {
                        "58:98:6f:00:0b:7a": ""
                    },
                },
                "severity": 3,
                "since": 9874
            }
        }
    }
}
```

At the time of this example request our Processor has been running for nearly four hours (X-RD-Uptime).

> X-RD-Uptime: 14232

Further down in the response there's an over temperature condition which was asserted at 9874 (since).

> "since": 9874

Time of response reception was roughly 5:28pm May 31st 2017 PDT.

> X-RD-Uptime – since = 14232 – 9874 = 4358 seconds ago

Meaning our alert first occurred at:

> 2017-05-31 17:28:00 – 4358 seconds => 4:15:22pm May 31st 2017 PDT (an hour ago from local time)

Note: Using uptime for the time base allows for events to be tied to a monotonic clock. This removes reliance on an accurate server clock, as there are many situations where the Processor may report time incorrectly (lack of Internet connectivity, flat battery, etc). As such it is discouraged to rely on the server Date response header.

8.2.4. Example - Over temperature alert (receiver card)

Condition: PSU and HUB board temperatures of an LED panel are over the maximum ratings

```
{
    "dev": {
        "receivers": {
            "RECEIVER_MAC_ADDRESS": {
                "alerts": {
                    "tempHigh": {
                        "desc":"",
                        "devices": {
                            "temps": { "CPU": "" }
                        }
                    }
                },
                "alertsCount": 1
            }
        }
    },
    "sys": {
        "alerts": {
            "tempHigh": {
                "desc":"",
                "devices": {
                    "receivers": {"RECEIVER_MAC_ADDRESS": ""}
                }
            }
        },
        "alertsCount": 1
    }
}
```

8.2.5. Example - Over temperature alert (Processor)

Condition: PSU temperature of the processor is over its maximum rating

```
{
    "dev": {
        "ingest": {
            "alerts": {
                "overTemp": {
                "desc":"",
                    "devices": {
                        "PSU": ""
                    }
                }
            }
        }
    },
    "sys": {
        "alerts": {
            "overTemp": {
                "desc": "",
                "devices": {
                    "ingest": ""
                }
            }
        },
        "alertsCount": 1
    }
}
```

8.2.6. Example - Port capacity (switch + receiver)

Condition: an LED panel could not be mapped because the Switch port capacity has been exceeded (~ port full).

```
{
    "dev": {
        "switches": {
            "SWITCH_MAC_ADDRESS": {
                "alerts": {
                    "portCap": {
                        "desc": "",
                        "devices": {
                            "ports": { "3": "" }
                        }
                    }
                },
                "alertsCount": 1
            }
        },
        "receivers": {
            "RECEIVER_MAC_ADDRESS": {
                "alerts": {
                    "portCap": {
                        "desc":""
                    }
                },
                "alertsCount": 1
            }
        }
    }
    "sys": {
        "alerts": {
            "portCap": {
                "desc": "",
                "devices": {
                    "receivers": {"RECEIVER_MAC_ADDRESS": ""},
                    "switches": {"SWITCH_MAC_ADDRESS": ""}
                }
            }
        },
        "alertsCount": 2
    }
}
```

9. Appendix

9.1. Data operations overview

Operation	REST	JSON-RPC
Reading property values	GET	"method": "state"
Writing property values	PATCH	"method": "set"
Performing actions	n/a	"method": "ping"
Alert / notifications	n/a	"method": "update"

9.2. Example - Blackout

9.2.1. Blackout

REST

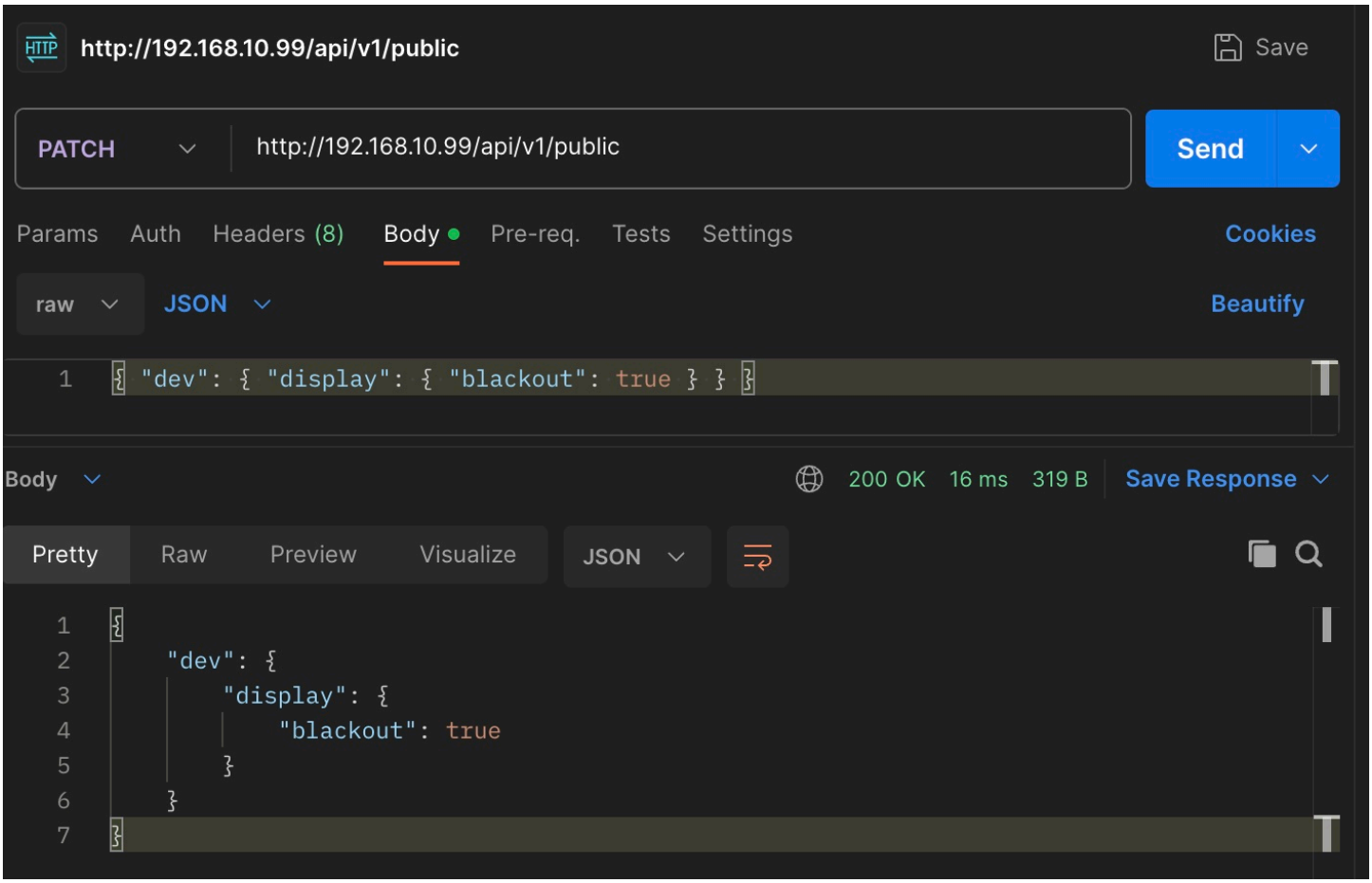
Request:

```
PATCH /api/v1/public
content-type: application/json

{ "dev": { "display": { "blackout": true } } }
```

Response:

```
{"dev": {"display": {"blackout": true}}}
```



9.2.2. Un-blackout

REST

Request:

```
PATCH /api/v1/public
content-type: application/json

{ "dev": { "display": { "blackout": false } } }
```

Response:

```
{"dev": {"display": {"blackout": false}}}
```

9.3. Example - Control screen brightness

9.3.1. Get screen brightness

REST

Request:

```
GET /api/v1/public?dev.display.brightness
```

Response:

```
{"dev": {"display": {"brightness": 28.200000762939453}}}
```

JSON-RPC

It is not possible to get an individual or specific property with JSON-RPC. You should get the current state using the 'state' method and then listen for updates every time the value changes.

Request:

```
{
    "jsonrpc": "2.0",
    "id": 100,
    "method": "state"
}
```

Response:

```
{
    "jsonrpc":"2.0",
    "id":100,
    "result": {"dev": {"display": {"brightness":28.200000762939453, "gamma":1.5}}, ... }}
}
```

Notification (async):

```
{
    "jsonrpc": "2.0",
    "method": "update",
    "params": {"dev": {"display": {"brightness": 33.29999923706055}}}
}
```

9.3.2. Set screen brightness

REST

Request:

```
PATCH /api/v1/public

{"dev": {"display": {"brightness": 50}}}
```

Response:

```
{"dev": {"display": {"brightness": 50}}}
```

JSON-RPC

Request:

```
{
    "jsonrpc": "2.0",
    "id": 101,
    "method": "set",
    "params": {"dev": {"display": {"brightness": 50}}}
}
```

Response:

```
{
    "jsonrpc": "2.0",
    "id": 101,
    "result":{"dev": {"display": {"brightness": 50}}}
}
```

9.4. Example - Control screen gamma

9.4.1. Get screen gamma

REST

Request:

```
GET /api/v1/public?dev.display.gamma
```

Response:

```
{"dev": {"display": {"gamma": 2.4}}}
```

JSON-RPC

It is not possible to get an individual or specific property with JSON-RPC. You should get the current state using the 'state' method and then listen for updates every time the value changes.

Request:

```
{
    "jsonrpc": "2.0",
    "id": 103,
    "method": "state"
}
```

Response:

```
{
    "jsonrpc": "2.0",
    "id": 103,
    "result": {"dev": {"display": {"brightness": 28.200000762939453, "gamma": 1.5}}, ... }}
}
```

Notification (async):

```
{
    "jsonrpc": "2.0",
    "method": "update",
    "params": {"dev": {"display": {"gamma": 2}}}
}
```

9.4.2. Set screen gamma

REST

Request:

```
PATCH /api/v1/public

{"dev": {"display": {"gamma": 2.1}}}
```

Response:

```
{"dev": {"display": {"gamma": 2.1}}}
```

JSON-RPC

Request:

```
{
    "jsonrpc": "2.0",
    "id": 104,
    "method": "set",
    "params": {"dev": {"display": {"gamma": 2.1}}}
}
```

Response:

```
{
    "jsonrpc": "2.0",
    "id": 104,
    "result": {"dev": {"display": {"gamma": 2.0999999046325684}}}
}
```

9.5. Example - Control test patterns

9.5.1. Enable color bars test pattern

REST

Request:

```
PATCH /api/v1/public

{
    "dev": {
        "ingest": {
            "testPattern": {
                "enabled": true,
                "type": "colorBars"
            }
        }
    }
}
```

Response:

```
{
    "dev": {
        "ingest": {
            "testPattern": {
                "enabled": true,
                "type": "colorBars"
            }
        }
    }
}
```

9.5.2. Show solid red test pattern

0xFF0000 => 16711680

REST

Request:

```
PATCH /api/v1/public

{
    "dev": {
        "ingest": {
            "testPattern": {
                "enabled": true,
                "color": 16711680,
                "type": "custom"
            }
        }
    }
}
```

Response:

```
{
    "dev": {
        "ingest": {
            "testPattern": {
                "enabled": true,
                "color": 16711680,
                "type": "custom"
            }
        }
    }
}
```

9.5.3. Disable test pattern

REST

Request:

```
PATCH /api/v1/public

{
    "dev": {
        "ingest": {
            "testPattern": {
                "enabled": false
            }
        }
    }
}
```

Response:

```
{
    "dev": {
        "ingest": {
            "testPattern": {
                "enabled": false
            }
        }
    }
}
```

9.6. Example - Control inputs

9.6.1. List available inputs

REST

Request:

```
GET /api/v1/public?dev.ingest.inputs
```

Response:

```
{
    "dev": {
        "ingest": {
            "inputs": {
                "dp: {
                    ...,
                    "freq": 0,
                    "h": 0,
                    "name": "DP",
                    "resolution": "No input",
                    "valid": false,
                    "w": 1920,
                    ...
                },
                "hdmi: {
                    ...,
                    "freq": 60,
                    "h": 1080,
                    "name": "HDMI",
                    "resolution": "1920x1080p60.00",
                    "valid": true,
                    "w": 1920,
                    ...
                },
                ...
            }
        }
    }
}
```

9.6.2. Current input

REST

Request:

```
GET /api/v1/public?dev.ingest.input
```

Response:

```
{
    "dev": {
        "ingest": {
            "input": "hdmi"
        }
    }
}
```

9.6.3. Select input

REST

Request:

```
PATCH /api/v1/public

{
    "dev": {
        "ingest": {
            "input": "dp"
        }
    }
}
```

Response:

```
{
    "dev": {
        "ingest": {
            "input": "dp"
        }
    }
}
```

9.7. Example - Redundancy flip to main/backup

9.7.1. Flip to main

REST

Request:

```
PATCH /api/v1/public

{
    "dev": {
        "display": {
            "redundancy": {
                "state": "main"
            }
        }
    }
}
```

Response:

```
{
    "dev": {
        "display": {
            "redundancy": {
                "state": "active"
            }
        }
    }
}
```

9.7.2. Flip to backup

REST

Request:

```
PATCH /api/v1/public

{
    "dev": {
        "display": {
            "redundancy": {
                "state": "backup"
            }
        }
    }
}
```

Response:

```
{
    "dev": {
        "display": {
            "redundancy": {
                "state": "active"
            }
        }
    }
}
```

9.8. Example - Groups

Note: "0" = first group

9.8.1. Group blackout

REST

Request:

```
PATCH /api/v1/public

{
    "dev": {
        "groups": {
            "0": {
                "blackout": true
            }
        }
    }
}
```

Response:

```
{
    "dev": {
        "groups": {
            "0": {
                "blackout": true
            }
        }
    }
}
```

9.8.2. Group 50% intensity

REST

Request:

```
PATCH /api/v1/public

{
    "dev": {
        "groups": {
            "0": {
                "gains": {
                    "i": 0.5
                }
            }
        }
    }
}
```

Response:

```
{
    "dev": {
        "groups": {
            "0": {
                "gains": {
                    "i": 0.5
                }
            }
        }
    }
}
```